

Laborprojekt – Versuch 9

Lernziele

Die Studierenden können einen Automaten korrekt erstellen und diesen in VHDL mithilfe einer PLA modellieren. Des Weiteren sind sie in der Lage, eine PLA zu kürzen und diese mit D-Flip-Flops und T-Flip-Flops umzusetzen.

Vorbereitung

In VHDL verwenden wir oft benutzerdefinierte Typen wie z. B. `state_type`, um Zustände oder Zustandsmaschinen zu modellieren (z. B. `S0`, `S1`, `S2`). Der aktuelle Zustand kann dann z. B. durch das Signal `current_state` und der nächste Zustand durch das Signal `next_state` repräsentiert werden.

```
type state_type is (S0, S1, S2);  
signal s_current_state, s_next_state : state_type;
```

Listing 1: Der Typ `state_type`

Beispiel

Stellen Sie sich vor, Sie müssen einen einfachen Ampelautomaten für einen Fußgängerüberweg in VHDL umsetzen. Der Automat hat drei Zustände:

- **Green:** Die Ampel zeigt grün für den Fußgänger.
- **Yellow:** Die Ampel wechselt zu gelb, um den Fußgänger auf das Ende der Grünphase vorzubereiten.
- **Red:** Die Ampel zeigt rot für den Fußgänger an.

Der Übergang zwischen den Zuständen soll mit einer Zeitverzögerung von jeweils 3 Sekunden erfolgen. Ein externes Signal P_CLK (mit einer Frequenz von 1 Hz) wird verwendet, um die Zeit zu messen. Die Zustandsübergänge erfolgen, wenn der Zähler diesen Wert erreicht.

1. Definieren der Zustände und VHDL-Typen

Zunächst erstellen wir einen benutzerdefinierten Typen `state_type`, der die möglichen Zustände des Automaten repräsentiert. Danach verwenden wir `current_state` und `next_state` als Signale, um den aktuellen und nächsten Zustand zu speichern.

```
-- Definition des benutzerdefinierten Typs
type state_type is (Green, Yellow, Red);

-- Signale für den aktuellen und nächsten Zustand
signal current_state, next_state : state_type;
signal counter : integer range 0 to 3 := 0; -- Zähler für die
      Zeit
```

Listing 2: Zustandsdefinition in VHDL

2. Der Zustandsübergang

Nun legen wir fest, wie die Zustände über die Zeit wechseln. In der Architektur müssen wir das Verhalten des Automaten unter Verwendung eines `process`-Blocks definieren. Wir berücksichtigen den Zähler und den Übergang von einem Zustand zum nächsten.

```
process(P_CLK)
begin
  if rising_edge(P_CLK) then
    -- Zähler erhöhen, um die Zeit zu messen
    if counter = 3 then
      counter <= 0;
      -- Übergang zum nächsten Zustand
      current_state <= next_state;
    else
      counter <= counter + 1;
    end if;
  end if;
end process;
```

Listing 3: Zustandsübergang in VHDL

3. Zustandsübergang und Logik

Nun definieren wir, wie der Zustand des Automaten basierend auf dem aktuellen Zustand und dem Zähler verändert wird. Wenn der Zähler den Wert 3 erreicht, wechselt der Automat zum nächsten Zustand.

```
process(current_state)
begin
    case current_state is
        -- Wenn wir im Zustand "Green" sind, wechseln wir nach
        -- 3 Sekunden zu "Yellow"
        when Green => next_state <= Yellow;
        -- Nach dem Zustand "Yellow" wechseln wir nach 3 Sekunden
        -- zu "Red"
        when Yellow => next_state <= Red;
        -- Nach dem Zustand "Red" wechseln wir nach 3 Sekunden
        -- zurück zu "Green"
        when Red => next_state <= Green;
        when others => next_state <= Green;
    end case;
end process;
```

Listing 4: Logik für die Zustandsübergänge

4. Ausgabe (Ampelsteuerung)

Zum Schluss definieren wir die Ausgänge der Ampel, die den aktuellen Zustand widerspiegeln. Der Ausgang P_TRAFFIC_LIGHT wird so gesetzt, dass die Ampel je nach Zustand grün, gelb oder rot anzeigt.

```
process(current_state)
begin
    case current_state is
        when Green  => P_TRAFFIC_LIGHT <= "00";  -- Grün
        when Yellow => P_TRAFFIC_LIGHT <= "01";  -- Gelb
        when Red    => P_TRAFFIC_LIGHT <= "10";  -- Rot
        when others => P_TRAFFIC_LIGHT <= "00";  -- Standard: Grün
    end case;
end process;
```

Listing 5: Ampelsteuerung in VHDL

5. Testbench und Simulation

Für die Testbench müssen Sie sicherstellen, dass Sie die Werte für P_CLK in regelmäßigen Abständen setzen, damit die Simulation korrekt funktioniert.

Beispiel-Testbench:

```
process
begin
    -- Testsignal für P_CLK erzeugen
    P_CLK <= '0';
    wait for 500 ms;
    P_CLK <= '1';
    wait for 500 ms;
    -- Wiederholen Sie dies, um den Timer zu simulieren
end process;
```

Listing 6: Testbench für die Simulation

Zusammenfassung

Zustandsdefinition: Erstellen eines benutzerdefinierten Typs `state_type`, um die Zustände des Automaten zu repräsentieren.

Automatenlogik: Verwendung eines `process`-Blocks, um den Zustand basierend auf dem Zähler und der Zeit zu wechseln.

Ausgabe: Steuerung der Ampel basierend auf dem aktuellen Zustand des Automaten.

Testbenches: Erstellen eines einfachen Testbenches, um den Automaten zu simulieren.

Aufgaben

Aufgabe 1: Bahnübergang

Setzen Sie die folgende Beschreibung eines einfachen Bahnübergangs als Automaten (Moore-Automat) in der Architektur `behavior` der Datei `my_rail_crossing.vhdl` um:

Ein Bahnübergang besitzt einen Sensor (`P_TRACK_OCCUPIED`), der erkennt, ob sich ein Zug im Gefahrenbereich des Bahnübergangs befindet. Führt ein Zug in den Gefahrenbereich des

Bahnübergangs ein ($P_TRACK_OCCUPIED = 1$), so schaltet zunächst die Ampel des Bahnübergangs auf rot ($P_TRAFFIC_LIGHT = 1$) und ein Timer von 5 Sekunden startet, bis sich die Schranken schließen ($P_GATE = 1$). Ist der Gefahrenbereich frei, so sollen sich sofort die Ampeln ausschalten und die Schranken öffnen.

Zustand	Codierung
Standby	00
Ampel rot	01
Schranke zu	10

Tabelle 1: Zustandskodierung

Hinweise zur Umsetzung:

- Stellen Sie den Automaten als Moore-Automat mit 3 Zuständen graphisch dar. Die Zustände finden sie in Tabelle 1
- Erstellen Sie einen Prozess, welcher auf die positive Taktflanke der Clock in den nächsten Zustand wechselt.
- Erstellen Sie einen weiteren Prozess in welchem sie in Abhängigkeit der Eingaben $S_CURRENT_STATE$, $P_TRACK_OCCUPIED$ und S_TIMER_OUT den neuen Zustand bestimmen in den bei einer positiven Taktflanke gewechselt wird.
- Gehen Sie davon aus, dass Ihnen am Port P_CLK_1HZ ein Signal mit der Frequenz 1 Hz bereitgestellt wird, welches Sie für den Timer verwenden sollen (dieses Signal wird für eine einfachere Simulation eventuell beschleunigt). Erstellen sie einen Prozess in welchem sie das Signal S_TIMER_INT mit jedem Durchlauf um 1 erhöhen um die Wartezeit darzustellen. Außerhalb des Timers darf P_CLK_1HZ nicht verwendet werden.
- Erstellen sie einen Prozess in welchem sie die Ausgaben $P_TRAFFIC_LIGHT$, P_GATE und S_TIMER_RUN in Abhängigkeit von $S_CURRENT_STATE$ bestimmen.
- Überprüfen Sie mit der Testbench `my_rail_crossing_tb_behavior.vhdl` Ihre Umsetzung erfolgreich.
- Erstellen Sie ein Bash-Script zur Automatisierung.
- Laden Sie die erstellten und angepassten Dateien, inklusive Bash-Script, als Zip-File `Name_Vorname_Versuch9_Aufgabe1.zip` in Moodle hoch.

Abnahme *Standard VHDL, Automaten*

Aufgabe 2: PLA.....

Geben Sie die vollständige Zustandsübergangstabelle und die vollständige Ausgabetabelle Ihres Automaten an. Stellen Sie anschließend Ihren Automaten in einer PLA dar und setzen Sie diese in VHDL als Dataflow in der Architektur `dataflow_full` um. Zur binären Codierung nutzen Sie Tabelle 1.

S_1	S_0	Track_OCCUPIED	Timer	S'_1	S'_0	TIMER_RUN	TRAFFIC_LIGHT	GATE
0	0	0	0	0	0	0	0	0
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

Tabelle 2: Zustandsübergangs- und Ausgabetabelle

Hinweise zur Umsetzung:

- Erstellen Sie die vollständige Zustandsübergangstabelle und Ausgabetabelle nach dem Schema der Tabelle 2.
- Stellen Sie Ihren Automaten in einer PLA dar. Nutzen sie für die Umsetzung des Speichers D-Flipflops. Sie sind dabei nicht an ihre Umsetzung in Versuch 6 gebunden.
- Setzen Sie die PLA in VHDL in der Architektur `dataflow_full` um (Der Timer und notwendige Flipflops dürfen weiterhin als behavior umgesetzt werden). Teilen Sie dafür Ihre PLA zwischen den Mintermen und der Disjunktion dieser und setzen Sie die Zeilen und Spalten jeweils getrennt um.
- Überprüfen Sie mit der Testbench `my_rail_crossing_tb_dataflow_full.vhdl` Ihre Umsetzung erfolgreich.
- Erstellen Sie ein Bash-Script zur Automatisierung.
- Laden Sie die erstellten und angepassten Dateien, inklusive Bash-Script, als Zip-File `Name_Vorname_Versuch9_Aufgabe2.zip` in Moodle hoch.

Abnahme *Standard VHDL, Testbench, PLA, Automaten*

Aufgabe 3: Minimieren.....

- Minimieren Sie ihre PLA auf 6 Zeilen und setzen sie diese minimierte PLA in VHDL als Dataflow in der Architektur `dataflow_min`, wie in Aufgabe 2 um.
- Überprüfen Sie mit der Testbench `my_rail_crossing_dataflow_min_tb.vhdl` Ihre Umsetzung erfolgreich.
- Erstellen Sie zudem ein Bash-Script zur Automatisierung
- Laden Sie die erstellten und angepassten Dateien, inklusive Bash-Script, als Zip-File `Name_Vorname_Versuch9_Aufgabe3.zip` in Moodle hoch.

Abnahme *Standard VHDL, Testbench, PLA, Minimierung*

Knobelaufgabe 4: T-Flipflops.....

- Setzen sie Aufgabe 3 mit Toggle-Flipflops, welche Sie in der Vorlesung kennenlernen durften, anstelle von D-Flipflops als Dataflow in der Architektur `dataflow_min_tff` um.
- Überprüfen Sie mit der Testbench `my_rail_crossing_tb_tff.vhdl` Ihre Umsetzung erfolgreich.
- Erstellen Sie ein Bash-Script zur Automatisierung
- Laden Sie die erstellten und angepassten Dateien, inklusive Bash-Script, als Zip-File `Name_Vorname_Versuch9_Aufgabe_Knobel.zip` in Moodle hoch.

Abnahme *Standard VHDL, T-Flipflop, Testbench*